

```

from pymongo import MongoClient
import sqlite3
import pandas as pd
import time
from pathlib import Path

```

```

CSV_PATH = Path(r"C:\Users\aj_m\Desktop\sales_data_sample.csv")

```

```

SOURCE_TABLE_CANDIDATES = ["final_etl_sales", "final_etl_table"]
RAW_TABLE = "raw_sales_data"
TRANSFORMED_TABLE = "final_elt_sales"
VALIDATION_TABLE = "elt_validation_report"
METRICS_TABLE = "elt_performance_metrics"
USD_TO_SAR_RATE = 3.75

```

```

REQUIRED_COLUMNS = [
    "ORDERNUMBER",
    "QUANTITYORDERED",
    "PRICEEACH",
    "SALES",
    "ORDERDATE",
    "STATUS",
    "QTR_ID",
    "MONTH_ID",
    "YEAR_ID",
    "PRODUCTLINE",
    "MSRP",
    "PRODUCTCODE",
    "CUSTOMERNAME",
    "CITY",
    "COUNTRY",
    "DEALSIZE",
]

```

```

def connect_database(db_path: Path) -> sqlite3.Connection:
    """Connect to SQLite database."""
    if not db_path.exists():
        raise FileNotFoundError(f"Database file not found: {db_path.resolve()}")
    return sqlite3.connect(db_path)

```

```

def get_existing_tables(conn: sqlite3.Connection) -> list[str]:
    """Return all user-created tables in the database."""
    query = "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;"
    return [row[0] for row in conn.execute(query).fetchall()]

```

```

def choose_source_table(conn: sqlite3.Connection) -> str:
    """Choose the available project table to use as the ELT source."""
    tables = get_existing_tables(conn)
    for table in SOURCE_TABLE_CANDIDATES:
        if table in tables:
            return table
    raise ValueError(f"No source table found. Available tables: {tables}")

```

```

def load_raw_data(conn: sqlite3.Connection, source_table: str) -> int:
    """
    ELT Load step:
    Copy the existing source table into a raw table without cleaning it first.
    """
    conn.execute(f"DROP TABLE IF EXISTS {RAW_TABLE};")
    conn.execute(f"CREATE TABLE {RAW_TABLE} AS SELECT * FROM {source_table};")
    conn.commit()
    return conn.execute(f"SELECT COUNT(*) FROM {RAW_TABLE};").fetchone()[0]

```

```

def run_validation_checks(conn: sqlite3.Connection) -> pd.DataFrame:
    """
    Data validation and quality checks.
    This is equivalent to GE-style expectations, but implemented directly in SQL
    so the file runs without extra installations.
    """
    checks = []

```

```

def add_check(check_name: str, sql_query: str, expected: str = "0 issues"):
    issue_count = conn.execute(sql_query).fetchone()[0]
    status = "PASS" if issue_count == 0 else "FAIL"
    checks.append(
        {
            "check_name": check_name,
            "issue_count": issue_count,

```

```

        "expected_result": expected,
        "status": status,
    }
)

add_check(
    "SALES should not be null",
    f"SELECT COUNT(*) FROM {RAW_TABLE} WHERE SALES IS NULL;",
)

add_check(
    "ORDERDATE should not be null",
    f"SELECT COUNT(*) FROM {RAW_TABLE} WHERE ORDERDATE IS NULL OR TRIM(ORDERDATE) = '';",
)

add_check(
    "SALES should be greater than zero",
    f"SELECT COUNT(*) FROM {RAW_TABLE} WHERE SALES <= 0 OR SALES IS NULL;",
)

add_check(
    "PRICEEACH should be greater than zero",
    f"SELECT COUNT(*) FROM {RAW_TABLE} WHERE PRICEEACH <= 0 OR PRICEEACH IS NULL;",
)

add_check(
    "QUANTITYORDERED should be greater than zero",
    f"SELECT COUNT(*) FROM {RAW_TABLE} WHERE QUANTITYORDERED <= 0 OR QUANTITYORDERED IS NULL;",
)

add_check(
    "Duplicate ORDERNUMBER values are allowed",
    "SELECT 0;"
)

add_check(
    "Important text columns should not be empty",
    f"""
SELECT COUNT(*) FROM {RAW_TABLE}
WHERE PRODUCTLINE IS NULL OR TRIM(PRODUCTLINE) = ''
   OR CUSTOMERNAME IS NULL OR TRIM(CUSTOMERNAME) = ''
   OR CITY IS NULL OR TRIM(CITY) = ''
   OR COUNTRY IS NULL OR TRIM(COUNTRY) = '';
""",
)

report_df = pd.DataFrame(checks)
report_df.to_sql(VALIDATION_TABLE, conn, if_exists="replace", index=False)
conn.commit()
return report_df


def transform_with_sql(conn: sqlite3.Connection) -> int:
    """
    ELT Transform step:
    Transform data inside SQLite using SQL after raw loading.
    """
    selected_columns = [col for col in REQUIRED_COLUMNS]
    columns_sql = ",\n".join(selected_columns)

    conn.execute(f"DROP TABLE IF EXISTS {TRANSFORMED_TABLE};")

    transform_sql = f"""
CREATE TABLE {TRANSFORMED_TABLE} AS
SELECT
    {columns_sql},
    ROUND(SALES * {USD_TO_SAR_RATE}, 2) AS SALES_SAR,
    ROUND(SALES / NULLIF(QUANTITYORDERED, 0), 2) AS SALES_PER_ITEM,
    UPPER(TRIM(CITY)) AS CITY_CLEAN,
    UPPER(TRIM(COUNTRY)) AS COUNTRY_CLEAN,
    UPPER(TRIM(PRODUCTLINE)) AS PRODUCTLINE_CLEAN,
    UPPER(TRIM(DEALSIZE)) AS DEALSIZE_CLEAN
FROM {RAW_TABLE}
WHERE SALES > 0
   AND PRICEEACH > 0
   AND QUANTITYORDERED > 0
   AND ORDERDATE IS NOT NULL
   AND TRIM(ORDERDATE) <> '';
"""
    conn.execute(transform_sql)
    conn.commit()
    return conn.execute(f"SELECT COUNT(*) FROM {TRANSFORMED_TABLE};").fetchone()[0]


def create_summary_tables(conn: sqlite3.Connection) -> None:
    """Create simple SQL aggregation tables for reporting and dashboard use."""
    conn.execute("DROP TABLE IF EXISTS elt_sales_by_productline;")
    conn.execute(
        f"""
CREATE TABLE elt_sales_by_productline AS
SELECT
    PRODUCTLINE_CLEAN AS PRODUCTLINE,
    COUNT(*) AS TOTAL_ORDERS,

```

```

        ROUND(SUM(SALES), 2) AS TOTAL_SALES_USD,
        ROUND(SUM(SALES_SAR), 2) AS TOTAL_SALES_SAR,
        ROUND(AVG(SALES), 2) AS AVG_SALES_USD
    FROM {TRANSFORMED_TABLE}
    GROUP BY PRODUCTLINE_CLEAN
    ORDER BY TOTAL_SALES_USD DESC;
    """
)

conn.execute("DROP TABLE IF EXISTS elt_sales_by_year;")
conn.execute(
    f"""
    CREATE TABLE elt_sales_by_year AS
    SELECT
        YEAR_ID,
        COUNT(*) AS TOTAL_ORDERS,
        ROUND(SUM(SALES), 2) AS TOTAL_SALES_USD,
        ROUND(SUM(SALES_SAR), 2) AS TOTAL_SALES_SAR
    FROM {TRANSFORMED_TABLE}
    GROUP BY YEAR_ID
    ORDER BY YEAR_ID;
    """
)
conn.commit()

```

```

def save_performance_metrics(
    conn: sqlite3.Connection,
    source_table: str,
    raw_rows: int,
    transformed_rows: int,
    elapsed_seconds: float,
) -> pd.DataFrame:
    """Save ELT performance metrics into SQLite."""
    metrics_df = pd.DataFrame(
        [
            {
                "pipeline_type": "ELT",
                "source_table": source_table,
                "raw_table": RAW_TABLE,
                "transformed_table": TRANSFORMED_TABLE,
                "raw_rows": raw_rows,
                "transformed_rows": transformed_rows,
                "removed_or_filtered_rows": raw_rows - transformed_rows,
                "execution_time_seconds": round(elapsed_seconds, 4),
                "execution_time_ms": round(elapsed_seconds * 1000, 2),
            }
        ]
    )
    metrics_df.to_sql(METRICS_TABLE, conn, if_exists="replace", index=False)
    conn.commit()
    return metrics_df

```

```

def export_to_mongodb(conn: sqlite3.Connection) -> None:
    """Export the transformed ELT table from SQLite to MongoDB."""
    client = MongoClient("mongodb://localhost:27017/")
    db = client["sales_project"]
    collection = db["final_elt_sales"]

    df = pd.read_sql_query(
        f"SELECT * FROM {TRANSFORMED_TABLE};",
        conn
    )

    collection.delete_many({})

    records = df.to_dict(orient="records")

    if records:
        collection.insert_many(records)

    print("\nData exported to MongoDB successfully.")
    print(f"MongoDB rows inserted: {len(records)}")

```

```

def main() -> None:
    start_time = time.perf_counter()

    conn = sqlite3.connect("sales_warehouse.db")

    df = pd.read_csv(
        r"C:\Users\aj_m\Desktop\sales_data_sample.csv",
        encoding="latin1"
    )

    df.to_sql("final_etl_sales", conn, if_exists="replace", index=False)

```

```

source_table = "final_etl_sales"

raw_rows = load_raw_data(conn, source_table)
validation_report = run_validation_checks(conn)
transformed_rows = transform_with_sql(conn)

create_summary_tables(conn)
export_to_mongodb(conn)

elapsed_seconds = time.perf_counter() - start_time

metrics = save_performance_metrics(
    conn,
    source_table,
    raw_rows,
    transformed_rows,
    elapsed_seconds
)

print("ELT pipeline completed successfully.")
print(f"Source table: {source_table}")
print(f"Raw table created: {RAW_TABLE} ({raw_rows} rows)")
print(f"Transformed table created: {TRANSFORMED_TABLE} ({transformed_rows} rows)")

print("\nValidation report:")
print(validation_report.to_string(index=False))

print("\nPerformance metrics:")
print(metrics.to_string(index=False))

print("\nCreated tables:")

for table in [
    RAW_TABLE,
    TRANSFORMED_TABLE,
    VALIDATION_TABLE,
    METRICS_TABLE,
    "elt_sales_by_productline",
    "elt_sales_by_year",
]:
    print(f"- {table}")

if __name__ == "__main__":
    main()

```